

 SOFTWARE ENGINEERING COURSE

LECTURE 3

Software Development Processes

Micheal Tuape, D.Sc

MODULE 1 SCHEDULE

Week	Dates	Topic	SE Chapters
Week 1	September 2 — September 6, 2024	Course Introduction	None
Week 2	September 9 — September 13, 2024	Introduction to Software Engineering	Ch. 1
Week 3	September 16 — September 20, 2024	Software Development Life Cycle & Methodologies	Ch. 2
Week 4	September 23 — September 28, 2024	Software Development Processes	Ch. 3
Week 5	October 2 — October 8, 2024	Requirements Engineering	Ch. 4
Week 6	October 9 — October 15, 2024	Software Modelling, Coding and Dev.	Ch. 5
Week 7	October 16 — October 22, 2024	Software Architectural Design	Ch. 6



LECTURE 2 AGENDA

- »» Software process models
- »» Process activities
- »» Coping with change
- »» Software Process improvement(SPI)





THE SOFTWARE PROCESS

- A **structured set of activities** required to develop a software system.
- Many different software processes but all involve:
 - **Specification** - defining what the system should do;
 - **Design and implementation** - defining the organization of the system and implementing the system;
 - **Validation** - checking that it does what the customer wants;
 - **Evolution** - changing the system in response to changing customer needs.
- A software process model is an **abstract representation** of a process. It presents a description of a process from some particular perspective.





SOFTWARE PROCESS DESCRIPTIONS

- When we describe and discuss processes, we usually talk about the **activities** in these processes such as specifying a data model, designing a user interface, etc. and the **ordering** of these activities.
- Process descriptions may also include:
 - **Products**, which are the outcomes of a process activity;
 - **Roles**, which reflect the responsibilities of the people involved in the process;
 - **Pre-** and **post-conditions**, which are statements that are true before and after a process activity has been enacted or a product produced.



PLAN-DRIVEN AND AGILE PROCESSES

- »» **Plan-driven processes** are processes where all of the process activities are planned in advance and progress is measured against this plan.
- »» In **agile processes**, planning is incremental and it is easier to change the process to reflect changing customer requirements.
- »» In practice, most practical processes include elements of both plan-driven and agile approaches.
- »» There are **no right or wrong software processes**.



SOFTWARE PROCESS MODELS

»»The waterfall model

- Plan-driven model. Separate and distinct phases of specification and development.

»»Incremental development

- Specification, development and validation are interleaved. May be plan-driven or agile.

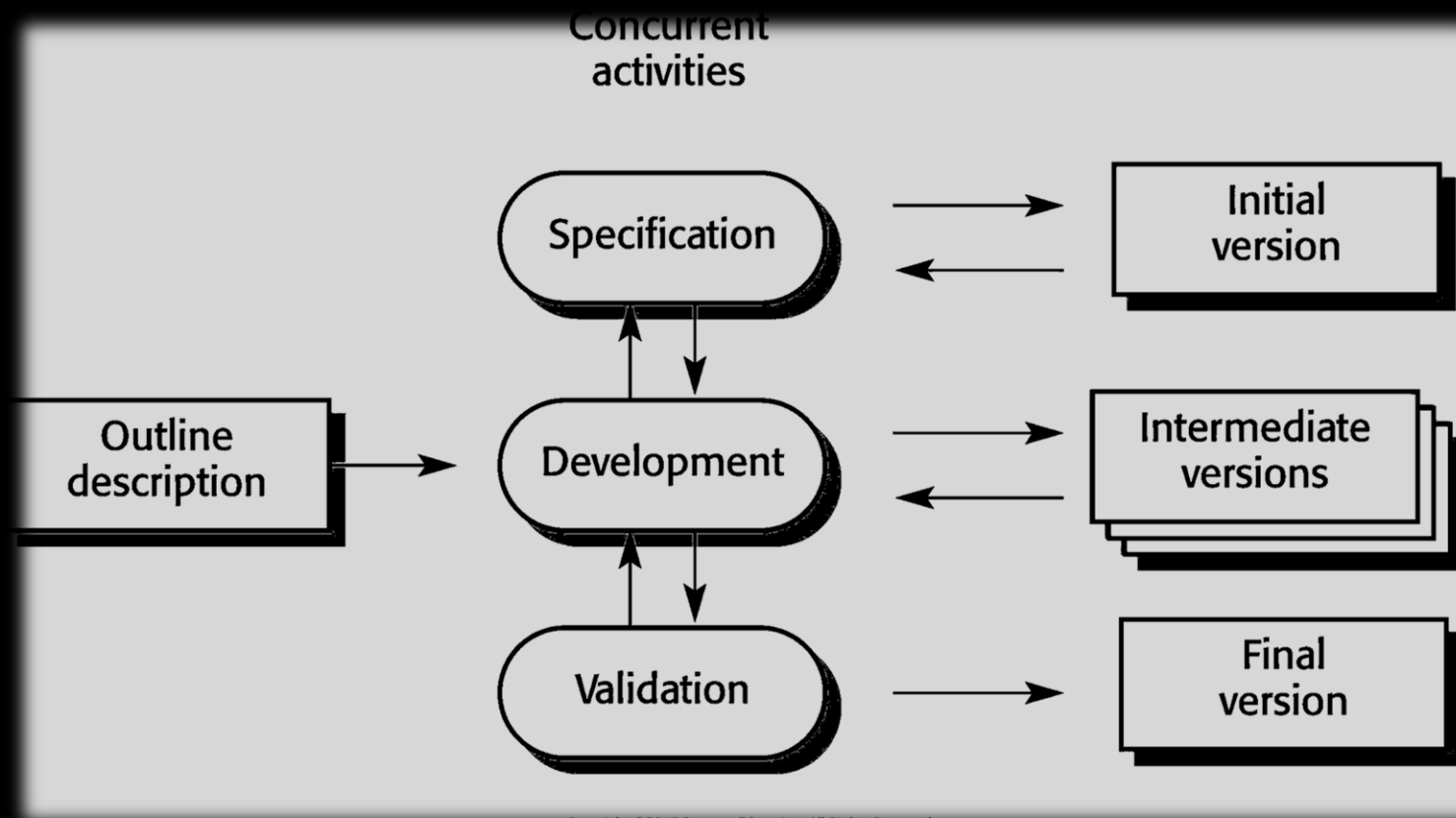
»»Integration and configuration

- The system is assembled from existing configurable components. May be plan-driven or agile.

- »»In practice, most large systems are developed using a process that incorporates elements from all of these models.



INCREMENTAL DEVELOPMENT





INCREMENTAL DEVELOPMENT BENEFITS

- The cost of **accommodating changing** customer requirements is reduced.
 - The amount of analysis and documentation that has to be redone is much less than is required with the waterfall model.
- It is easier to get **customer feedback** on the development work that has been done.
 - Customers can comment on demonstrations of the software and see how much has been implemented.
- More **rapid delivery and deployment** of useful software to the customer is possible.
 - Customers are able to use and gain value from the software earlier than is possible with a waterfall process.



INCREMENTAL DEVELOPMENT PROBLEMS

- »» The process is **not visible**.
 - Managers need regular deliverables to measure progress. If systems are developed quickly, it is not cost-effective to produce documents that reflect every version of the system.
- »» System structure tends to **degrade** as new increments are added.
 - Unless time and money is spent on refactoring to improve the software, regular change tends to corrupt its structure. Incorporating further software changes becomes increasingly difficult and costly.

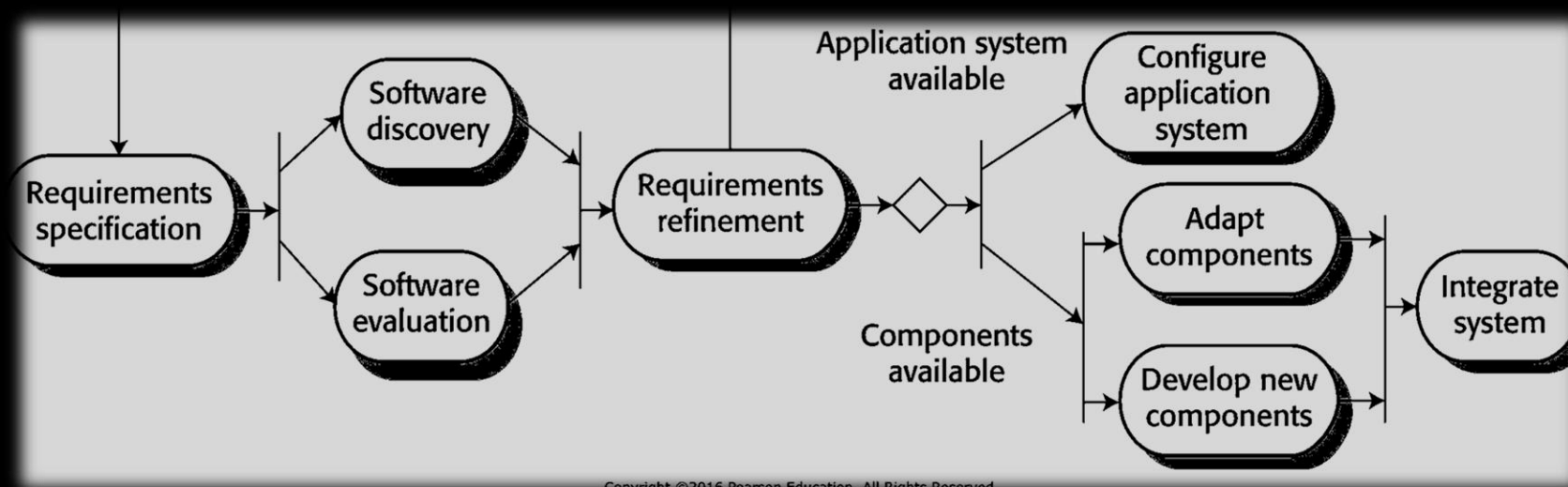


INTEGRATION AND CONFIGURATION

- »» Based on **software reuse** where systems are integrated from **existing** components or application systems (sometimes called C O T S - Commercial-off-the-shelf) systems).
- »» Reused elements may be **configured** to adapt their behaviour and functionality to a user's requirements
- »» Reuse is now the standard approach for building many types of business system



REUSE-ORIENTED SOFTWARE ENGINEERING



Copyright ©2016 Pearson Education, All Rights Reserved





ADVANTAGES AND DISADVANTAGES

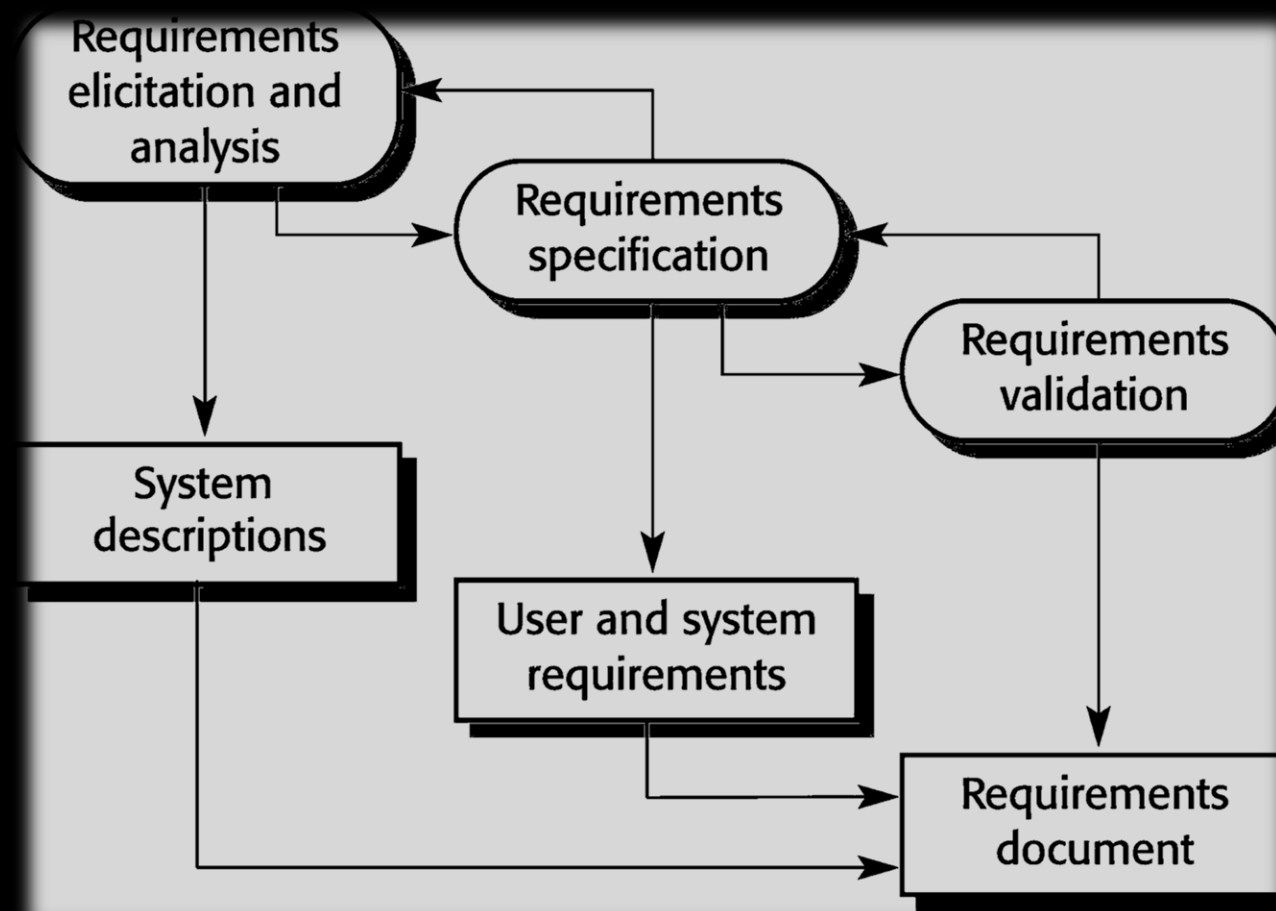
- »» Reduced costs and risks as less software is developed from scratch
- »» Faster delivery and deployment of system
- »» But requirements compromises are inevitable so system may not meet real needs of users
- »» Loss of control over evolution of reused system elements



PROCESS ACTIVITIES

- »» Real software processes are **inter-leaved sequences** of technical, collaborative and managerial activities with the overall goal of specifying, designing, implementing and testing a software system.
- »» The four basic process activities of **specification**, **development**, **validation** and **evolution** are organized differently in different development processes.
- »» For example, in the waterfall model, they are organized in sequence, whereas in incremental development they are interleaved.

THE REQUIREMENTS ENGINEERING PROCESS





SOFTWARE SPECIFICATION

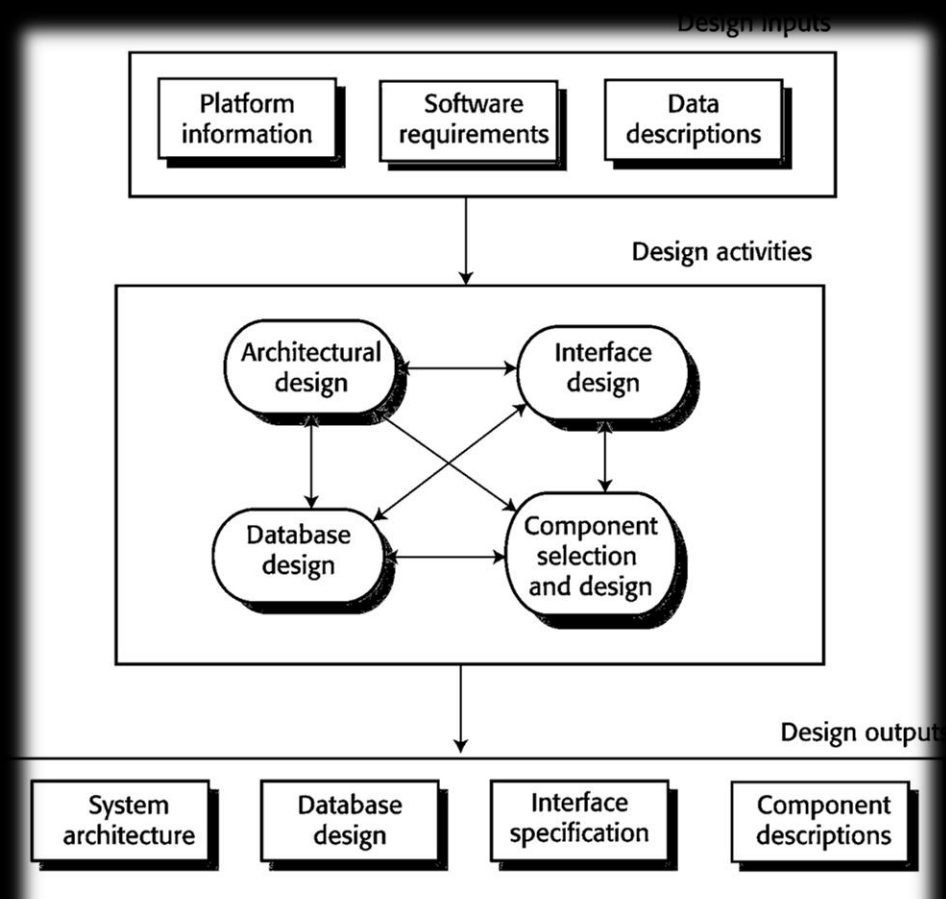
- »» The process of establishing what services are required and the constraints on the system's operation and development.
- »» Requirements engineering process
 - Requirements elicitation and analysis
 - What do the system stakeholders require or expect from the system?
 - Requirements specification
 - Defining the requirements in detail
 - Requirements validation
 - Checking the validity of the requirements



SOFTWARE DESIGN AND IMPLEMENTATION

- »» The process of converting the system specification into an executable system.
- »» **Software design**
 - Design a software structure that realises the specification;
- »» **Implementation**
 - Translate this structure into an executable program;
- »» The activities of design and implementation are closely related and may be inter-leaved.

A GENERAL MODEL OF THE DESIGN PROCESS





SYSTEM IMPLEMENTATION

- »» The software is implemented by
 - developing a program or programs
 - configuring an application system.

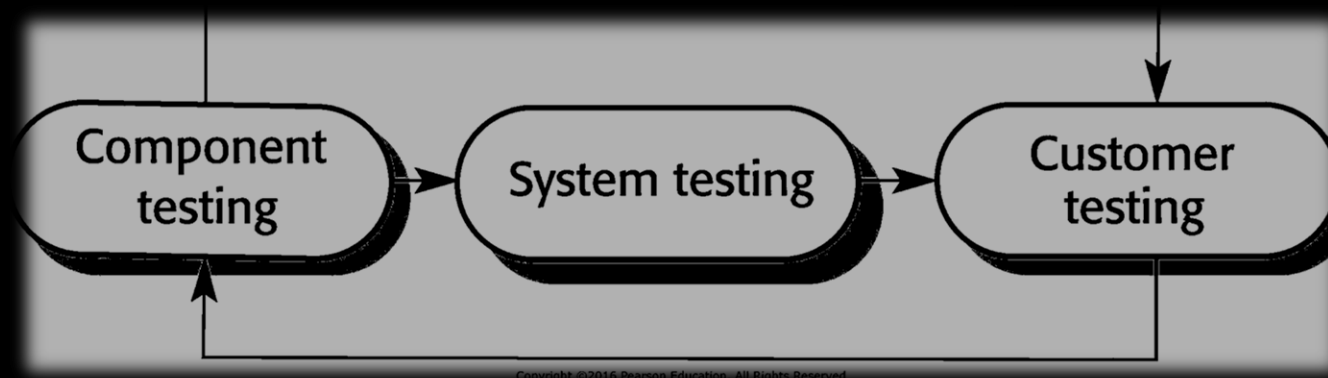
- »» Design and implementation are **interleaved** activities for most types of software system.



SOFTWARE VALIDATION

- »» **Verification** and **Validation** is intended to show that a system conforms to its specification and meets the requirements of the system customer.
- »» Involves checking and review processes and system testing.
- »» System testing involves executing the system with test cases that are derived from the specification of the real data to be processed by the system.
- »» **Testing** is the most commonly used Verification and Validation activity.

STAGES OF TESTING

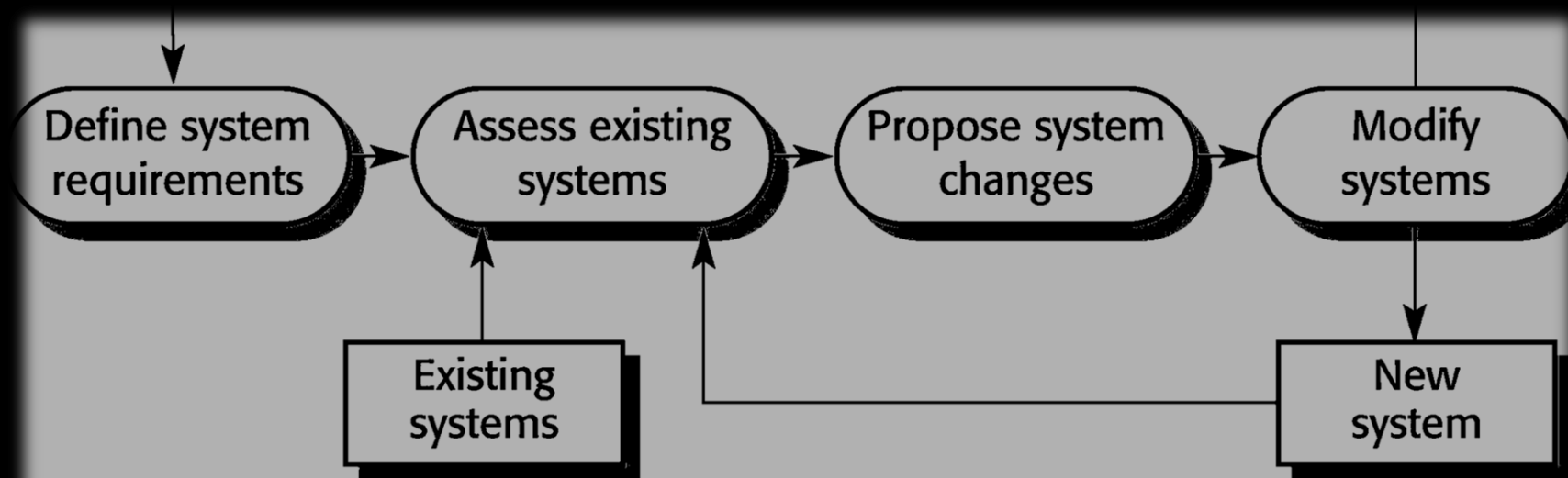




SOFTWARE EVOLUTION

- »» Software is inherently **flexible** and can **change**.
- »» As requirements change through changing business circumstances, the software that supports the business **must also evolve and change**.
- »» Although there has been a demarcation between development and evolution (maintenance) this is increasingly irrelevant as fewer and fewer systems are completely new.

SOFTWARE SYSTEM EVOLUTION



Copyright ©2016 Pearson Education, All Rights Reserved



COPING WITH CHANGE

- »» Change is **inevitable** in all large software projects.
 - Business changes lead to new and changed system requirements
 - New technologies open up new possibilities for improving implementations
 - Changing platforms require application changes
- »» Change leads to rework so the costs of change include both rework as well as the costs of implementing new functionality



REDUCING THE COSTS OF REWORK

- »» **Change anticipation**, where the software process includes activities that can anticipate possible changes before significant rework is required.
- »» **Change tolerance**, where the process is designed so that changes can be accommodated at relatively **low cost**.



COPING WITH CHANGING REQUIREMENTS

- **System prototyping**, where a version of the system or part of the system is developed quickly to check the customer's requirements and the feasibility of design decisions. This approach supports change anticipation.
- **Incremental delivery**, where system increments are delivered to the customer for comment and experimentation. This supports both change avoidance and change tolerance.



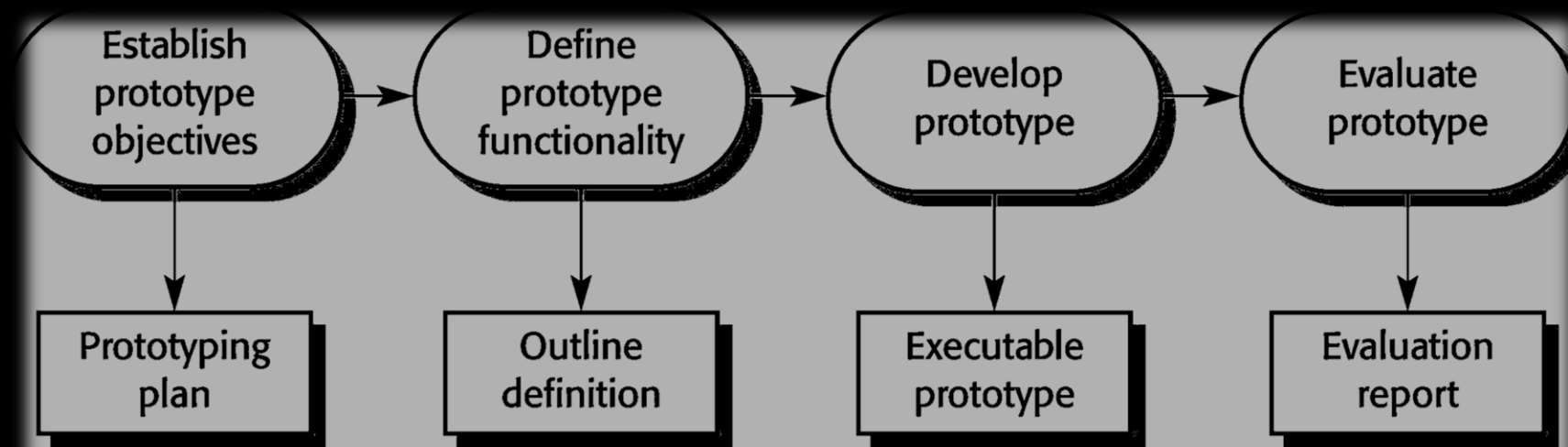
SOFTWARE PROTOTYPING

»» A **prototype** is an **initial version** of a system used to demonstrate concepts and try out design options.

»» Benefits of Prototyping

- Improved system usability.
- A closer match to users' real needs.
- Improved design quality.
- Improved maintainability.
- Reduced development effort.

PROTOTYPE DEVELOPMENT



Copyright ©2016 Pearson Education, All Rights Reserved



INCREMENTAL DELIVERY

- Rather than deliver the system as a single delivery, the development and delivery is broken down into **increments** with each increment delivering part of the required functionality.
- User requirements are **prioritised** and the highest priority requirements are included in early increments.
- Once the development of an increment is started, the requirements are frozen though requirements for later increments can continue to evolve.



INCREMENTAL DELIVERY ADVANTAGES

- »» Customer value can be delivered with each increment so system functionality is **available earlier**.
- »» Early increments act as a **prototype** to help elicit requirements for later increments.
- »» **Lower risk** of overall project failure.
- »» The highest priority system services tend to receive the most testing.



INCREMENTAL DELIVERY PROBLEMS

- Most systems require a **set of basic facilities** that are used by different parts of the system.
 - It can be hard to identify common facilities that are needed by all increments.
- The essence of iterative processes is that the specification is **developed in conjunction with the software**.
 - This conflicts with the procurement model of many organizations, where the complete system specification is part of the system development contract.



PROCESS IMPROVEMENT

- »» Most of the Software companies large, medium, small, or startup usually face **issues in their software development projects** and its delivery. The issues can vary from lack of **documentation**, lack of **following the process**, lack of **process governance**, lack of the **integration and collaboration** between the teams, lack of **requirements traceability**, lack of **technology management**, ...etc
- »» This causes project failure Schedule, Cost, Quality .



PROCESS IMPROVEMENT

- »» SPI can be considered as a process of **re-engineering** or **change management** in a **software development project** to detect the software development **lifecycle inefficiencies** and resolve them to have a **better process**.



PROCESS IMPROVEMENT

- »» Many software companies have turned to software process improvement as a way of **enhancing the quality** of their software, **reducing costs** or **accelerating the development**.
- »» Process improvement means **understanding** existing processes and **changing** these processes to increase product quality and/or reduce costs and development time.



MOTIVATION FOR PROCESS IMPROVEMENT

Standardization and Process consistency

- » To have a standard and practical process for software development mapped to organization goals and strategy.

Cost Reduction

- » To improve projects cost by enhancing the process and eliminate issues, redundancies, and deficiencies.

Competitive Edge

- » Being certified in CMMI for example, can put the company in higher competitive edge and make it gain more sales due to the evidence of existing mature software process based on standard method.

Meeting targets and reduce time to market

- » Meeting organization goals, projects delivery, quality standards, valuable products, professional documentation are outputs from SPI.



MOTIVATION FOR PROCESS IMPROVEMENT

Improve customers satisfaction

- » Project delivery on time and based on the specification with high quality will improve customers satisfaction and improve the sales process.

Job satisfaction

- » Employees get job satisfaction from producing a good quality product and knowing what to do without workload and the time consumed to resolve conflicts or to eliminate issue due to an immature process.

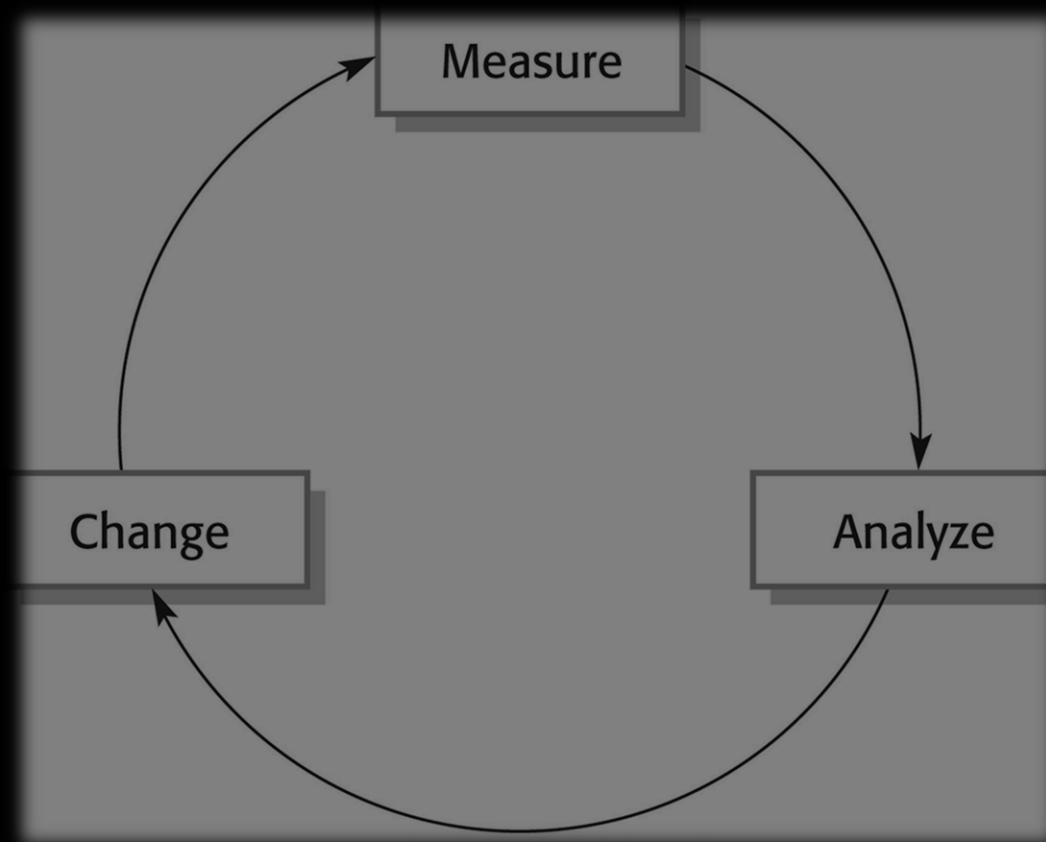
Automation and Autonomy

- » Introducing tools to automate things and improve quality and ensure consistency. Moreover, enabling different employees to play different roles in the project.

Proven outcome

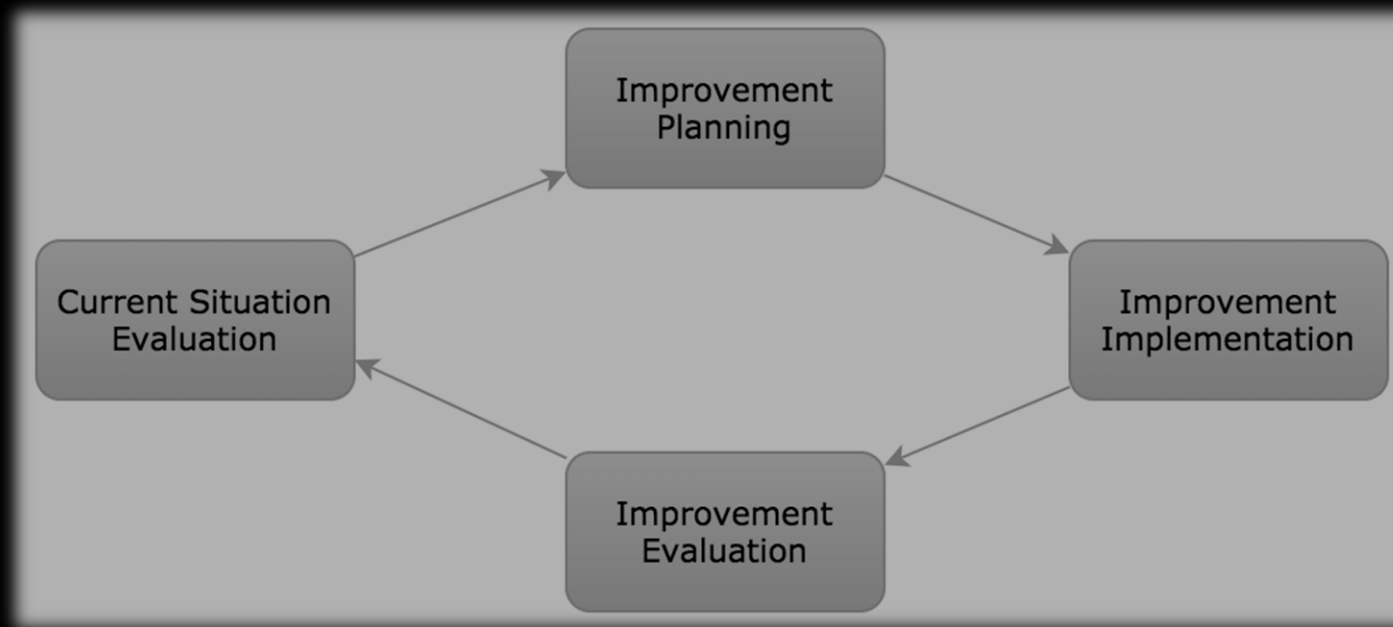
- » There is a lot of evidence for the value of SPI projects which are successfully implemented.

THE PROCESS IMPROVEMENT CYCLE



Copyright ©2016 Pearson Education. All Rights Reserved

THE PROCESS IMPROVEMENT CYCLE





PROCESS MEASUREMENT

- »» Wherever possible, **quantitative process data** should be collected
 - However, where organisations do not have clearly defined process standards this is very difficult as you don't know what to measure. A process may have to be defined before any measurement is possible.
- »» Process measurements should be used to **assess** process improvements
 - But this does not mean that measurements should drive the improvements. The improvement driver should be the organizational objectives.



PROCESS METRICS

- »» Time taken for process activities to be completed
 - **Examples:** Calendar time or effort to complete an activity or process.
- »» Resources required for processes or activities
 - **Examples:** Total effort in person-days.
- »» Number of occurrences of a particular event
 - **Examples:** Number of defects discovered.



PROCESS IMPROVEMENT

- »» Similar to the SDLC, SPI has a lot of methods and you can as well define your own method if it is effective or combine between more than one if you do not have any preferences or organization need to adopt a specific method.
- »» SPI methods mainly categorized into two categories: inductive (bottom-up approach) and prescriptive (top-down approach) and most of them are cyclic with four main steps.



PROCESS IMPROVEMENT

» SPI methods mainly categorized into two categories: inductive (bottom-up approach) and prescriptive (top-down approach) and most of them are cyclic with four main steps.

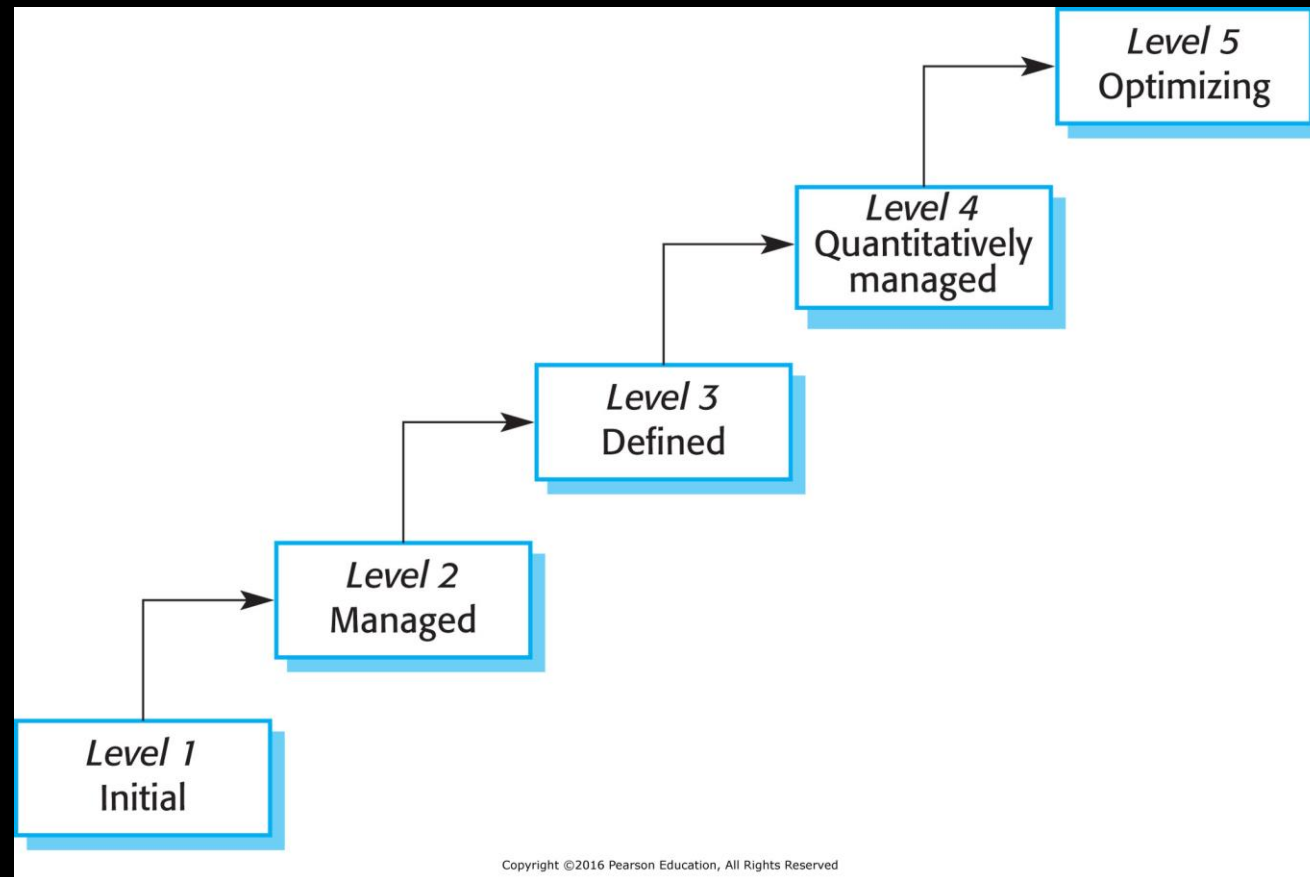
Inductive

- Quality Improvement Paradigm (QIP)
- Improvement framework utilizing lightweight assessment and improvement planning (iFLAP)

Prescriptive

- Capability Maturity Model Integration (CMMI)
- Software Process Improvement and Capability dEtermination (SPICE)

CAPABILITY MATURITY LEVELS





CAPABILITY MATURITY LEVELS

Read about

Kanban

Crystal

Capability Maturity Model Integration (CMMI)

Software Process Improvement and Capability dEtermination (SPICE)

